CH4 VHDL Language

4.1 What is VHDL?

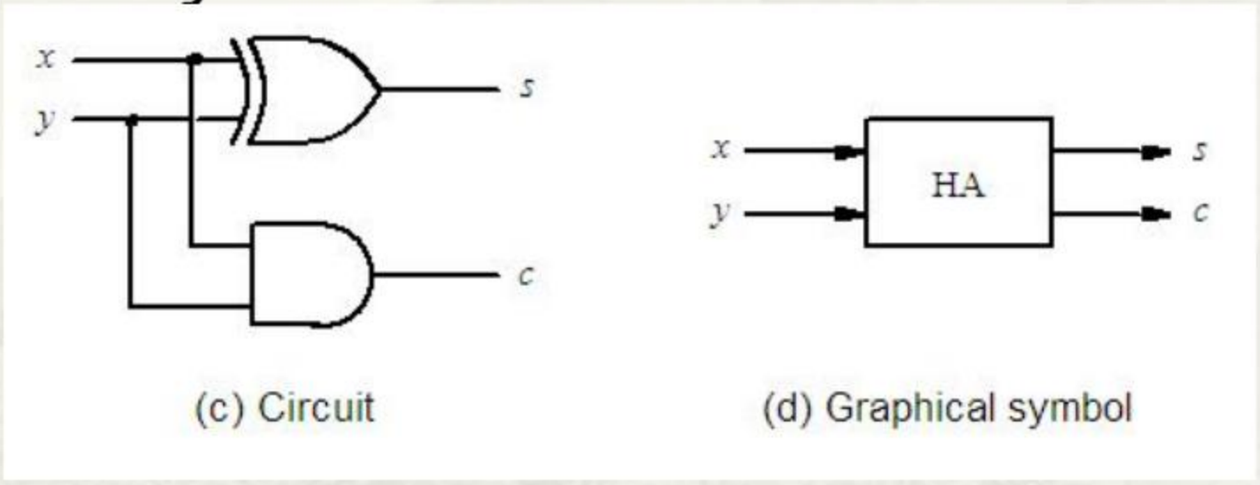
- VHDL: Very high speed integrated circuits Hardware Description Language.
- An industry standard of IEEE & U.S. DoD used to model digital systems at many levels.
- VHDL 87 & VHDL 93
- A core subset is to be studied.

4.2 Design Units

- Entity
- Architecture
- *Configuration*
- Library
- Package

4.2.1 Entity Declaration

- Specify the device name and external interfaces.



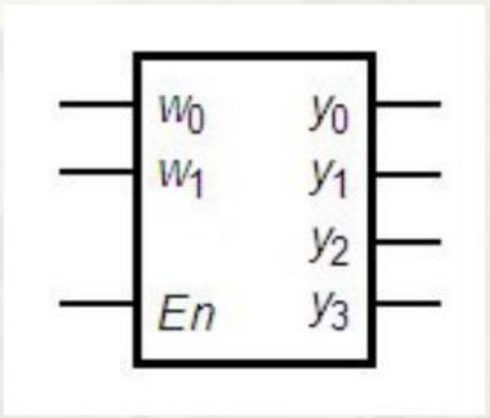
(c) Circuit (d) Graphical symbol

```
ENTITY half_adder IS
 PORT (x, y: IN BIT;     --signal name, mode & type
       s, c: OUT BIT);
END half_adder;
-- This is a comment line
```

4 Modes for VHDL signals

- IN: an input port
- OUT: an output port
- *INOUT*
- *BUFFER: to be detailed in latter chapters.*

Another example

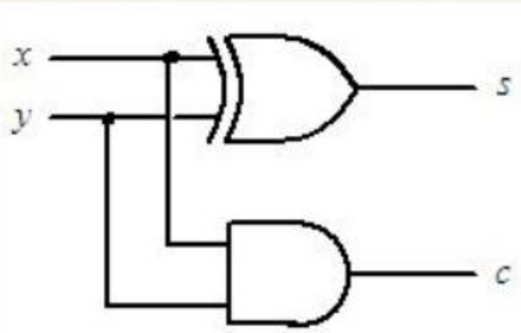



```
ENTITY dec2to4 IS                                    --a 2-to-4 decoder
   PORT(w1,w0,en :IN BIT;
            y  :OUT BIT_VECTOR(3 DOWNTO 0)  --one of  array types
           );
END dec2to4;
```

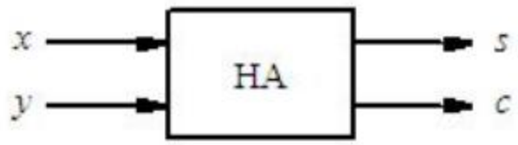
4.2.2 Architecture Body

- Specify the internal details with 1 of following styles:
  - Dataflow: A set of concurrent assignment statements.
  - Behavioral: A set of sequential assignment statements .
  - Structural: A set of interconnected components.

1. Dataflow



(c) Circuit



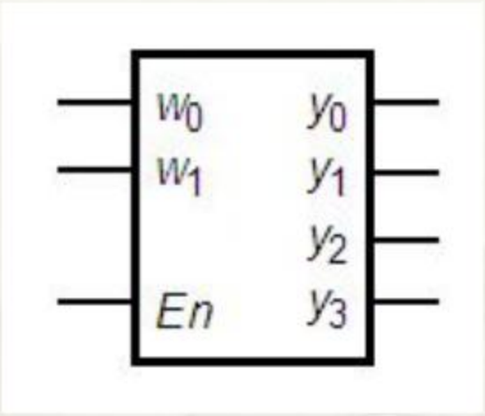
(d) Graphical symbol

--E.g.1 : dataflow description of HA

```
ARCHITECTURE ha_concurrent OF half_adder IS  --start of ARC
--Declarative part in between
BEGIN
    s <= x XOR y;      --2 concurrent signal assignment statements in arbitrary order
    c <= x AND y;
END ha_concurrent;
```

--E.g.2 : dataflow description of dec2to4

```
ARCHITECTURE behavior OF dec2to4 IS
BEGIN
    y(0) <= en AND NOT (w(1) OR w(0));
    y(1) <= en AND (NOT w(1)) AND w(0);
    y(2) <= en AND w(1) AND (NOT w(0));
    y(3) <= en AND w(1) AND w(0));
END ha_concurrent;
```

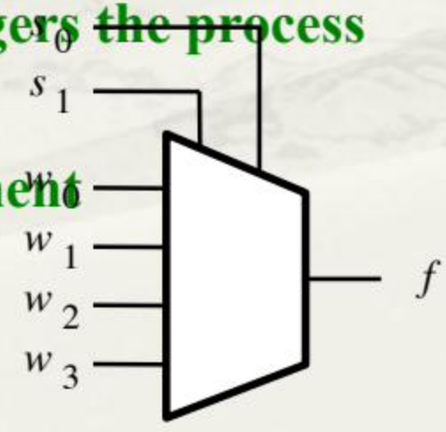


2. Behavioral

- Specify the internal details with a set of statements to be executed sequentially encapsulated inside a process statement.

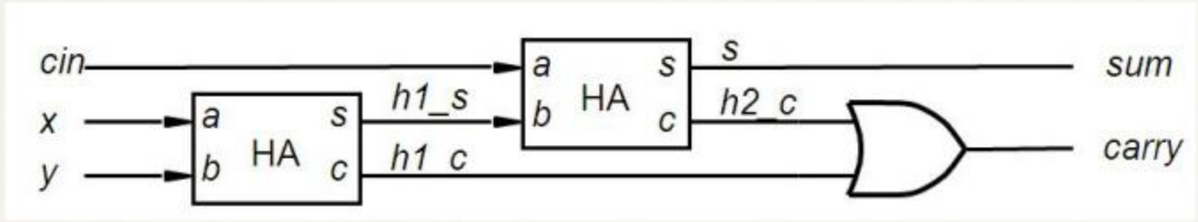
--E.g.: behavioral description of 4-to-1 multiplexer

```
ENTITY mux4to1 IS
    PORT(w :IN BIT_VECTOR(3 DOWNTO 0);
           s :  IN BIT_VECTOR(1 DOWNTO 0);
           f:   OUT BIT);
END mux4to1;
ARCHITECTURE  behave OF mux4to1 IS
BEGIN
PROCESS(s, w)     --process statement followed by a sensitivity list
BEGIN             --any time an event on any of signals in the list triggers the process
   CASE s IS
       WHEN "00" =>f<=w(0);    --sequential signal assignment statement
       WHEN "01" =>f<=w(1);
       WHEN "10" =>f<=w(2);
       WHEN "11" =>f<=w(3);
```



3. Structural

- An entity(high level) is described as a set of interconnected components(low level).
  - Every component should be implemented in advance.
  - Declare components in the declarative part of architecture body.
  - Link instantiated components to build the top-level circuit.



--E.g. of structural modeling: implementation of FA using HAs as components

```
ENTITY full_adder IS
  PORT (x,y,cin:IN BIT;          --interfaces of FA
        sum,carry:OUT BIT);
END full_adder;
ARCHITECTURE f_adder OF full_adder IS
  COMPONENT half_adder          --HA has its own entity and architecture having been realized
    PORT(a,b:IN BIT;            --copied from the PORT clause in HA's entity declaration
         s,c:OUT BIT);
  END COMPONENT;
SINGAL h1_s,h1_c,h2_c:BIT;      --internal signals in FA
BEGIN
 --2 PORT MAP clauses used to instantiate 2 components
  h1:half_adder PORT MAP(a=>x, b=>y, s=>h1_s, c=>h1_c);      --mappings in parenthesis
```

4.3 Identifiers

- Case insensitive
- Composed of letters, digits or '_'
- Beginning with a letter is required.
- Ending with '_' is illegal.
- "_ _"is illegal.

4.4 Data Objects

4.4.1. constant declarations

```
CONSTANT rise_time: TIME := 10ns;
CONSTANT bus_width: INTEGER := 8;
```

4.4.2. variable declarations

```
VARIABLE ctrl_status: BIT_VECTOR(10 DOWNTO 0);
VARIABLE sum: INTEGER RANGE 0 to 100 := 10;
VARIABLE found, done: BOOLEAN;
```

4.4.3. signal declarations

```
SINGAL h1_s,h1_c,h2_c:BIT;
SIGNAL data_bus: BIT_VECTOR(0 TO 7);
SIGNAL gate_delay: TIME := 10 NS;
```

Note: Interfaces declared in PORT clause are signals, where the keyword SIGNAL is optional .

Variables Vs. Signals

- Signals are global, variables are local.
  - Signals are declared outside subprogram and can be used anywhere in architecture body.
  - Variables can be declared and used only inside a subprogram.
  - ‘<=‘ for signals; ‘:=‘ for variables
  - Delayed value assignment for signals, instantaneous one for variables.

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY xor_sig IS
    PORT (a,b,c: IN std_logic;
                x,y :  OUT std_logic);
END xor_sig;
ARCHITECTURE sig_arch OF xor_sig  IS
   SIGNAL d:std_logic;
BEGIN
    sig :PROCESS(a,b,c)
    BEGIN
       d<=a;
       x<=c XOR d;
       d<=b;
       y<=c XOR d;
```

Result

x= c XOR b

y= c XOR b

If d is a var instead:

x= c XOR a

4.5 Operators

Logical	Relational	Arithmetical	Catenation
AND	=	+	
OR	/=	-	
NOT	<	*	
NAND	<= (also for signal assignment)	/	&
NOR	>	**(exponentiation)	
XOR	>=	MOD	
XNOR		REM	
		ABS	

4.6 Data Types

4.6.1 Part of Predefined **Data Types**

- BIT(0,1)
- BIT_VECTOR -- predefined array type
- INTEGER -- -(2^31 - 1) ~ (2^31 - 1)
- BOOLEAN(TRUE, FALSE)
- CHARACTER
- ...

4.6.1 Part of Predefined Data Types(cont.)

- **STD_LOGIC --One of the most frequently used types**

```
(   'U', -- Uninitialized
    'X', -- Forcing  Unknown
    '0', -- Forcing  0
    '1', -- Forcing  1
    'Z', -- High Impedance
    'W', -- Weak    Unknown
    'L', -- Weak    0
    'H', -- Weak    1
    '-'  -- Don't care
)
```

4.6.2 User Defined Data Types(self-studying)

- Enumeration types: anyone of which has a set of user-defined values consisting of identifiers and character literals.
- E.g.:

TYPE bit_logic IS (‘0’,’1’,’Z’,’X’);

TYPE traffic_light IS (red, green, yellow);

TYPE micro_op IS (load, store, add, sub, mul, div);

4.6.2 User Defined Data Types(cont.)

- Array types: consists of elements of the same type.
- E.g.:

```
TYPE  register IS  ARRAY(0 TO 7) OF BIT;
TYPE  register_1 IS  ARRAY(7 DOWNTO 0) OF BIT;
TYPE rom IS  ARRAY(0 TO 7) OF register;
TYPE rom IS  ARRAY(0 TO 7,0 TO 7) OF BIT;
TYPE std_logic_vector IS ARRAY(natural range<>)OF std_logic
```

4.7 Libraries and Packages

- A library includes packages.
- A package encapsulates a set of related declarations of:
  - types
  - constants
  - functions
  - subcircuits(components)
  - ...

4.7.1 Examples of packages in certain libraries

```
package STANDARD is          --in library STD
type BOOLEAN is (FALSE, TRUE);       --enumeration type
type BIT is ('0', '1');
type CHARACTER IS (…);
type INTEGER is range -2147483648 to 2147483647;
type BIT_VECTOR is array (NATURAL range <>) of BIT;
…
end STANDARD;
```

4.7.1 Examples of packages in certain libraries(cont.)

```
PACKAGE std_logic_1164 IS      --in library IEEE
...
SUBTYPE std_logic IS resolved std_ulogic;
TYPE std_logic_vector IS ARRAY ( NATURAL RANGE <>) OF std_logic;
FUNCTION "and"  ( l, r : std_logic_vector  ) RETURN std_logic_vector;
FUNCTION "nand" ( l, r : std_logic_vector  ) RETURN std_logic_vector;
FUNCTION "or"   ( l, r : std_logic_vector  ) RETURN std_logic_vector;
FUNCTION "nor"  ( l, r : std_logic_vector  ) RETURN std_logic_vector;
FUNCTION "xor"  ( l, r : std_logic_vector  ) RETURN std_logic_vector;
FUNCTION "xnor"  ( l, r : std_logic_vector  ) RETURN std_logic_vector;
FUNCTION "not"  ( l : std_logic_vector  ) RETURN std_logic_vector;
```

4.7.2 Declarations of libraries and packages

- Syntax as follows:

LIBRARY library-name;

e.g.: LIBRARY IEEE;

USE library-name. package-name. Item ;

e.g.: USE IEEE.STD_LOGIC_1164.ALL;

By the way

- Declaration of library STD is implicit.
- Declaration of library IEEE is explicit.

4.8 VHDL statements

4.8.1 Concurrent statements

- 1. Signal assignment statement

```
ARCHITECTURE ha_concurrent OF half_adder IS
BEGIN
    s <= a XOR b;
    c <= a AND b;
END ha_concurrent;
```

```
ARCHITECTURE behavior OF dec2to4 IS
BEGIN
    y(0) <= en AND NOT (w1 OR w(0));
    y(1) <= en AND (NOT w(1)) AND w(0);
    y(2) <= en AND w(1) AND (NOT w(0));
    y(3) <= en AND w(1) AND w(0));
END ha_concurrent;
```

2. Conditional signal assignment(when...else clauses)

Syntax:

Target - signal <= [value 1 when condition 1else]
[value 2 when condition 2 else]
. . .
value n;

```
ENTITY mux4to1 IS
   PORT(w : IN BIT_VECTOR(3 DOWNTO 0);
        s :  IN BIT_VECTOR(1 DOWNTO 0);
        f:   OUT BIT);
END mux4to1;
ARCHITECTURE  behave  OF mux4to1 IS
BEGIN
      f<=w(0) when s="00" else
         w(1) when s="01" else
         w(2) when s="10" else
```

Note:

- When…else must enumerate all the conditions.

```
--implement 4-to-1 multiplexer using when-else clauses
Library IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY mux4to1 IS
    PORT(w :IN STD_LOGIC_VECTOR(3 DOWNTO 0);
         s :  IN STD_LOGIC_VECTOR(1 DOWNTO 0);
         f:   OUT STD_LOGIC);
END mux4to1;
ARCHITECTURE  behave  OF mux4to1 IS
BEGIN
      f<=w(0) when s="00" else
         w(1) when s="01" else
         w(2) when s="10" else
```

3. Selected Signal Assignment Statement(with...select)

Syntax:

with expression select **--using all valuations as select conditions**

target-signal <= value 1 when condition 1,
value 2 when condition 2,
…
value n when condition n,
[value n+1 when others];

--Implement 4-to-1 multiplexer using with-select clauses

```
ARCHITECTURE  behave  OF mux4to1 IS
BEGIN
 WITH s SELECT
    y<=w(0) WHEN  “00”,
         w(1) WHEN  “01”,
         w(2) WHEN  “10”,
         w(3) WHEN  “11” ,
         ‘X’ WHEN OTHERS;
END behave;
```

4.8.2 Sequential statements

- 1. PROCESS statement

Syntax:

[process-label:]PROCESS[(sensitivity-list)]

[process-item-declarations]

begin

various kinds of sequential-statements

end process [process-label];

```
ARCHITECTURE ha_concurrent OF half_adder IS
BEGIN
    PROCESS(a,b)                --process statement followed by a sensitivity list
    BEGIN
        s <= a XOR b;           -- Statements inside process run sequentially
        c <= a AND b;
    END PROCESS;
END ha_concurrent;
```

2. Signal assignment statement

```
ARCHITECTURE ha_concurrent OF half_adder IS
BEGIN
    PROCESS(a,b)
    BEGIN
        s <= a XOR b;          -- Signal assignment statements inside process are sequential
        c <= a AND b;
    END PROCESS;
END ha_concurrent;
```

3. Variable assignment statement

3. IF statement

Syntax:

IF boolean-expression 1 THEN

sequential-statements

[ELSIF boolean-expression 2 then -- elsif clause; if stmt can have 0 or more elsif clauses.

sequential-statements]

…

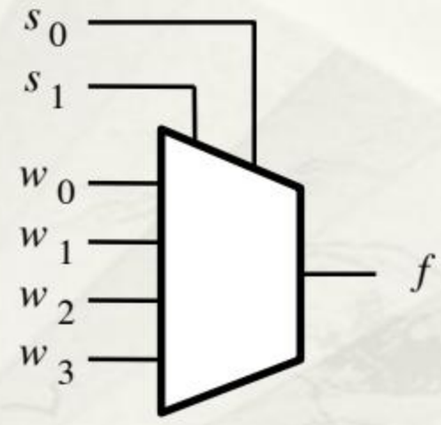
[ELSE -- else clause.

sequential-statements]

END IF;

IF statement can only be used in behavioral description!

```
--Implement 4-to-1 multiplexer using IF-ELSE statement
ARCHITECTURE  behave  OF mux4to1 IS
BEGIN
    PROCESS ( w, s )
        BEGIN
            IF s =“00” THEN
                f <= w(0) ;
            ELSIF s =“01” THEN
                f <= w(1) ;
            ELSIF s =“10” THEN
                f <= w(2) ;
            ELSE
                f <= w(3) ;
            END IF ;
END PROCESS ;
```



4. CASE statement

Syntax:

CASE expression IS

WHEN choice 1 => sequential-statements -- branch #1

WHEN choice 2=> sequential-statements -- branch #2 ***-- Any number of branches***

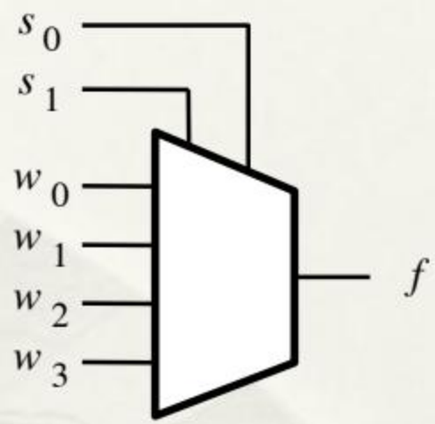
…

[WHEN OTHERS => sequential-statements] -- last branch if necessary

CASE statement can only be used in behavioral description!

END CASE;

```
--Implement 4-to-1 multiplexer using CASE statement
ARCHITECTURE  behave OF mux4to1 IS
BEGIN
PROCESS(s,w)
BEGIN
    CASE s IS                                   --suppose s is of type BIT
        WHEN "00" =>f<=w(0);
        WHEN "01" =>f<=w(1);
        WHEN "10" =>f<=w(2);
        WHEN "11" =>f<=w(3);            --here the branch “WHEN OTHERS” is optional
    END CASE;
END PROCESS;
END behave;
```



5. LOOP statement

Syntax:

[loop-label :] iteration-scheme LOOP ***--Types of iteration-schemes***

sequential-statements

END LOOP [loop-label] ;

Syntax of FOR :

[loop-label :] FOR identifier in range LOOP ***--Type 1 : FOR LOOP***

sequential-statements

END LOOP [loop-label] ;

Syntax of WHILE :

[loop-label :] WHILE boolean-expression LOOP ***--Type 2 : WHILE LOOP***

sequential-statements

END LOOP [loop-label] ;

--E.g. 1 of LOOP statement

…

FACTORIAL := 1; --variable

FOR number in 2 to N loop --number used without explicit declaration

FACTORIAL := FACTORIAL * NUMBER;

end loop; --number++ implicitly

--E.g. 2 of LOOP statement

…

J:=0;SUM:=10; --variables

WH-LOOP: while J < 20 loop --This loop has a label, WH_LOOP.

SUM := SUM * 2;

J:=J+3;

end loop WH-LOOP;

4.8.3 A comprehensive example:

Implementation of a 7-majority voting

```
Library ieee;
Use ieee.std_logic_1164.all;
Use ieee.std_logic_unsigned.all;
Entity vote7 is
  port (men: IN std_logic_vector(6 downto 0) ;   --let element value 1 □ voting for
        Vote_result:  OUT std_logic);            --let value 1 □voted through
End vote7;
Architecture vote of vote7 is
  signal pass: std_logic;
  begin
  Process (men)
    variable temp :std_logic_vector(2 downto 0);--set a counter to record votes
```

```
begin
    temp:="000";
    For i in 0 to 6 loop            --loop vars without declarations in advance
        if (men(i)='1') then
            temp:=temp+1;
        end if;
    end loop;
     pass<=temp(2);                 -- determine whether votes reach 4
   End Process;
     Vote_result<=pass;             -- pass the result to the output interface
End vote;
```